Verslag Geavanceerde Computerarchitectuur

Ybe Vandamme, Jarno De Jonckheere

2024



1 Labo 1: Getting Acquainted with GPU Programming

In dit labo was het de bedoeling om ons kennis te laten maken met GPU programmering. GPU programmering zorgt ervoor dat threads parallel kunnen worden uitgevoerd. De focus was dus om de structuur van het programmeren onder te knie te krijgen.
In dit labo werd er ons gevraagd om een rij in omgekeerde volgorde te bekomen. Dit kon op verschillende manieren. We konden dit doen a.d.h.v. 1 block met daarin 9 threads. Hierin zou het dus lijken op CPU programmering waarin elke thread om de beurt werd uitgevoerd. Maar we konden dit ook doen door 9 blocks met daarin elk 1 thread. Hierbij werd elke thread dus op een aparte Core parallel uitgevoerd. Hierin hebben we dus kunnen experimenteren en kijken wat de effecten zijn. Hieronder zie je een foto van de uitgevoerde code waarbij ook een timestamp werd gegeven om de snelheid waar te nemen van de verschillende testopstellingen.

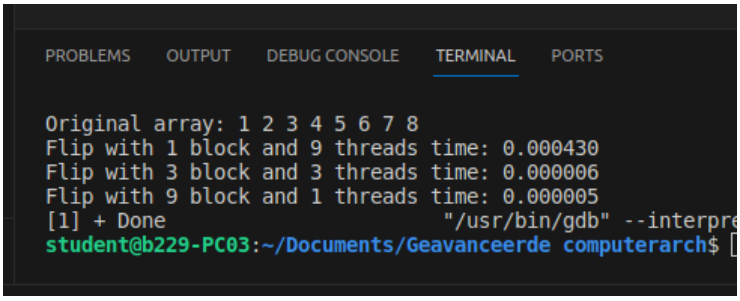


Figure 1: Output array flip

Hierop zien we dus dat het zeker een effect heeft om dingen parallel te laten lopen. Er is een gigantische tijdswinst door met parallelle threads te werken.

2 Labo 2: Race conditions

In dit labo was het de bedoeling om verschillende max-detectie algoritmes te implementeren en deze met elkaar te vergelijken. De drie verchillende implementaties die terug te vinden zijn in ons experiment zijn:

- Max via een for-loop op de CPU
- Max via atomic operations op de GPU
- Max via een reduction-algoritme op de GPU

In afbeelding 2 ziet u de resultaten van ons experiment. Merkwaardig is hierbij dat het reductie-algoritme het slechtst presteert. Dit is te verklaren door het feit dat de executietijd van een reductie-algoritme sterk afhankelijk is van de implementatie. Verder is het logisch dat het algoritme met atomic operations slechter presteert dan het algoritme op de CPU. Dit is te verklaren door het feit dat hier slechts één thread per keer toegang krijgt tot de array bijkomend met het feit dat de CPU geoptimaliseerd is voor sequentiële operaties, terwijl de GPU geoptimaliseerd is voor parallelle operaties. ten laatste kan men ook zeggen dat de extra overhead gecreëerd door het afsluiten/toekennen van toegang tot de array ook zorgt voor extra vertragingen.

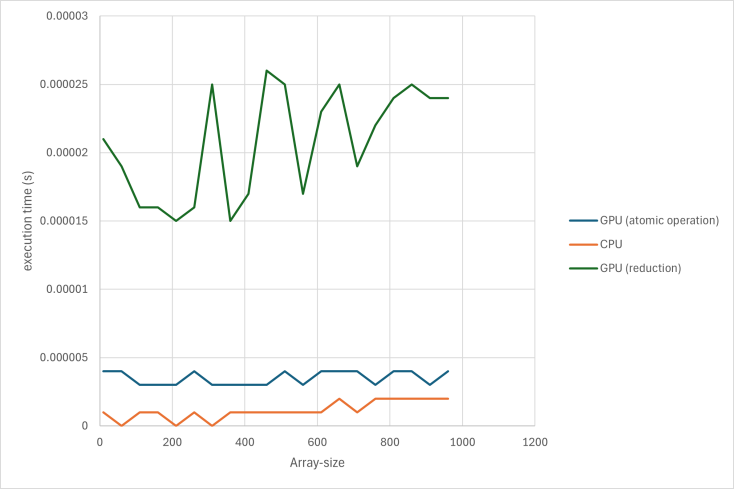


Figure 2: Resultaten experiment

Indien er minder threads voorzien worden dan elementen in de input-array dan zou onze kernel, zoals deze nu geschreven is, niet alle elementen processen.

2.1 Vragen

Q: **Is het resultaat correct?** Onze resultaten waren bij alle drie de manieren hetzelfde dus gaan we ervan uit dat deze juist zijn. Dit kun je zien in de OUPUT file die is meegeleverd.

Q: **Hoe veranderd de tijd in functie van de input grootte?**

Zie figuur 2.

Q: **Wat gebeurt er als de input groter is dan 2048 (hoe zorg je ervoor dat de correcte berekening wordt gedaan en is er een extra stap nodig)?**

We kunnen de functie atomic max gebruiken. Deze splits de array in blokken en haalt daaruit het maximum. Een andere manier is om de reductie te gebruiken om steeds kleiner te gaan.

Q: **Stel je voor dat je de kernel aanroept met veel minder threads dan er data is. Hoe zorg je ervoor dat alle data wordt geprocessed.**

We kunnen de rij splitsen om maximum aantal data te kunnen processen. Hierachter kunnen we die functie opnieuw oproepen om uit alle maximum waardes het maximum te halen.

Q: **Hoe zou je een 2D rij oplossen?** Je zou kunnen het maximum bepalen per rij of kolom en die waardes in een nieuwe array steken. Deze nieuwe array zou dan een 1D array zijn en zouden we dus met die voorbeelden hierboven kunnen verder verwerken.

3 Labo 3: warps, thread divergence and striding

3.1 PART 1: Warps

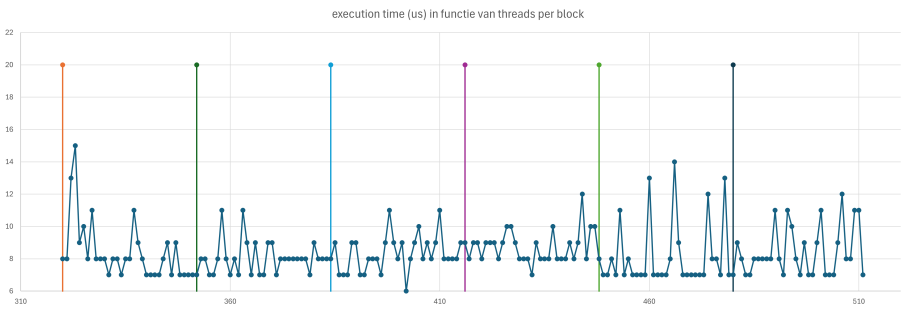


Figure 3: output van de standaard flip kernel

In afbeelding 3 ziet u het resultaat van het uitvoeren van de standaard flip kernel op de afbeelding. De horizontale lijnen zijn de plaatsen waar de de warps volledig gevuld zijn. Ter hoogte van de volle warps zouden we een betere prestatie moeten waarnemen. Dat dit nu moeilijk te besluiten is uit de afbeelding kan verklaard worden door het feit dat onze kernel een relatief lage computationele complexiteit heeft.Toch presteerd de kernel ter hoogte van de full warps toch duidelijk relatief goed.

De geschreven kernel blijkt te werken want deze resulteert in de negatieve versie in van de originele afbeelding. De geschreven kernel werkt ook indien er veel minder threads gescheduled worden ten opzichte van het aantal input-elementen met behulp van striding.

3.2 PART 2: Thread divergence

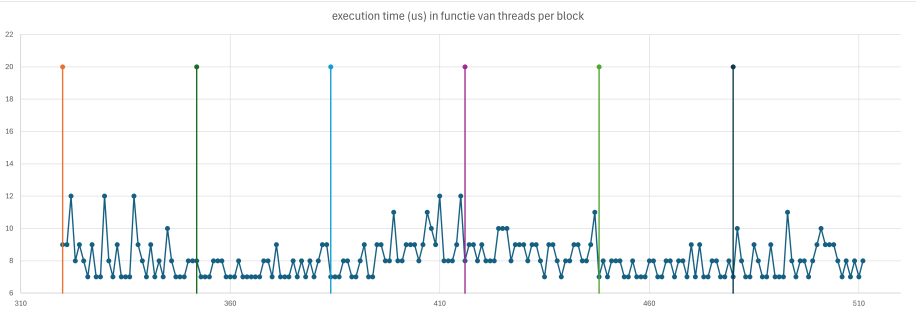


Figure 4: output van de red only flip kernel

In afbeelding 4 merken we op dat de prestatie niet aanzienlijk verbeterd ondanks het feit dat slechts een derde van het aantal input-elementen moet geflipped worden. Dit is te verklaren door het feit dat ook de threads die geen element moeten flippen gescheduled worden. Deze threads zijn idle tot de threads die wel degelijk een input-element moeten flippen klaar zijn. men zou kunnen kijken indien, het processen van de array van de vorm [RGB RGB ...] naar [RRR... GGG... BBB...] en vervolgens de kernel laten doorvoeren op het eerste deel (RRR), een betere executietijd oplevert. Dit zal voor complexere kernels zeker het geval zijn omdat nu minder strides nodig zijn maar zal hier waarschijnlijk niet duidelijk zichtbaar zijn.

3.3 Extra besluiten

We merken verder op dat soms de eerste executie van een kernel veel langer kan duren dan normaal en dat men deze meting best verwaarloosd.

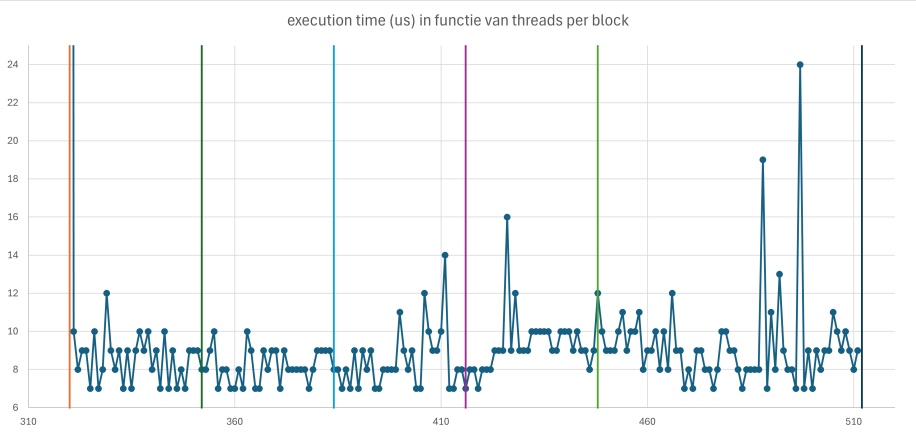


Figure 5: output van de kernel na processing naar [RRR... GGG... BBB...]

De kernel uit afbeelding 5 presteert niet beter dan de andere kernels omdat men hier nog niet werkt met coalesced memory acces.

4 Labo 4: Memory management

4.1 Part 1: Coalesced memory access

Bij coalesced memory access wordt met één transactie meerdere datapunten uit het geheugen opgehaald, in plaats van dat voor elk datapunt afzonderlijk een transactie nodig is. Binnen een warp kan dit bijvoorbeeld betekenen dat er slechts één keer data opgehaald wordt in plaats van 32 keer. Dit vermindert het aantal benodigde transacties, wat de uitvoertijd verbetert. De opdracht was om alleen de R-waarden van een afbeelding aan te passen, het resultaat daarvan vindt u hieronder.



(a) Originele foto



(b) Inverse R-waarde

Figure 6: Input en output

De resultaten die we hebben verkregen rond de uitvoer tijden, is samengevat in een grafiek in figuur 7.

4.1.1 Questions

- What is the performance difference between the coalesced and uncoalescs kernel?

  Uit figuur 7 stellen we vast dat coalesced vaak beter presteert. Dit komt omdat er minder bewerkingen nodig zijn en dus de kernel veel sneller kan gaan. Alle data die nodig is voor bewerking volgt elkaar op in memory wat dus zorgt voor hogere snelheid.

- How does the performance difference change dependent on input and/or grid size?

  In figuur 7 kan gezien worden dat bij uncoalesced de executietijd bij grotere grid sizes toeneemt, terwijl bij coalesced het juist lichtjes daalt.

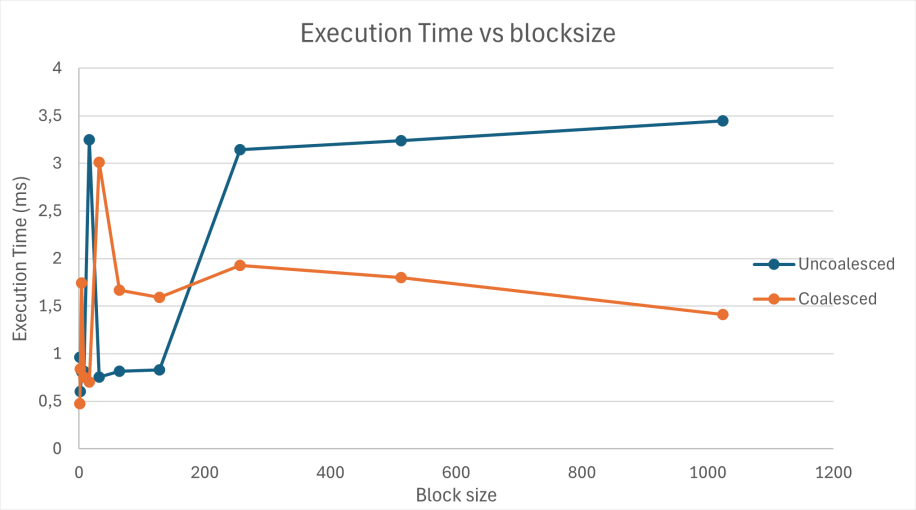


Figure 7: Grafiek execution time

4.2 Part 2: Global, shared, and constant memory

In deel 2 moesten we bestuderen hoe verschillende methoden van memory acces de snelheid bepalen van een wiskundige bewerking, in dit geval een matrix vermenigvuldiging. Bij gebruik van Global memory kwamen we uit op een gemiddelde van 249 ms, Shared een gemiddelde van 15 ms en constant memory kwamen we een gemiddelde van 22 ms. We kunnen uit deze resultaten besluiten dat shared memory het snelste type geheugen is dat kan gebruikt worden door de kernel op de GPU. Verder hebben we ook geëxperimenteerd van met de grootte van de n X n matrices en daarbij ook steeds de grootte van het grid aangepast. deze resultaten vind u terug in figuur 8

4.2.1 Questions

- How do the execution times of matrix multiplication compare when employing the three different memory types?

  We zien dat global memory altijd de langste tijd zal vergen, daarna komt constant memory en het snelste is shared memory.

- How does the execution times of matrix multiplication compare when employing the three different memory types?

  We zien dat execution time bij shared memory vrij constant blijft. Bij Global memory stijgt de execution time naarmate de grootte toeneemt. Ook constant memory heeft een grotere execution time naarmate de grootte toeneemt. Een duidelijke trend kunnen we niet echt zien wegens het beperkt aantal metingen die zijn gedaan.

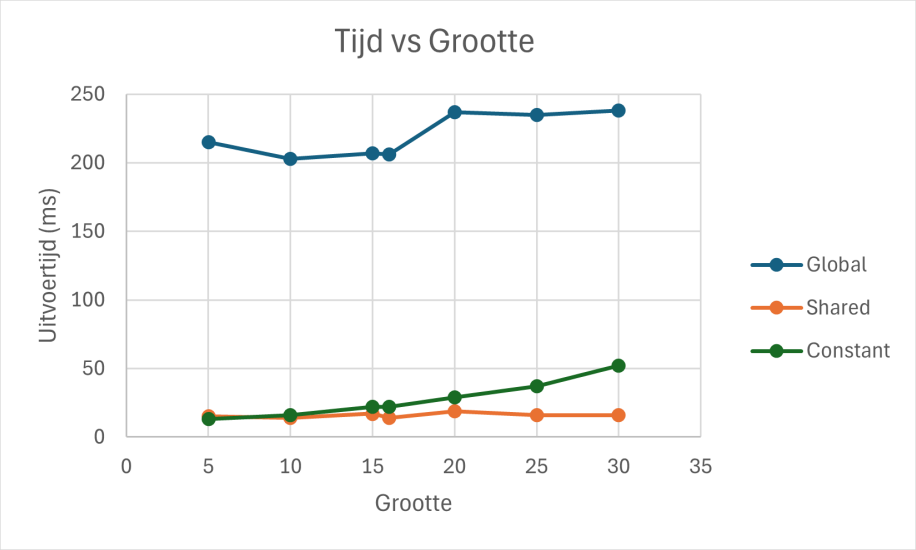


Figure 8: Grafiek Matrix

.

5 Labo 5: Dynamic parallelism and asynchronous operation

Deze labozitting was het de bedoeling om vier arrays aan te maken en vervolgens het maximum, het minimum, het product van de elementen en de som van de elementen te vinden. Dit alles hebben we op twee verschillende manieren gerealiseerd. In eerste instantie laten we de array-creatie en GPU-processing synchroon gebeuren. Vervolgens in onze tweede versie van het programma laten we GPU-processing van de ene array en het aanmaken van de volgende array telkens parallel verlopen.



Figure 9: Experiment 1

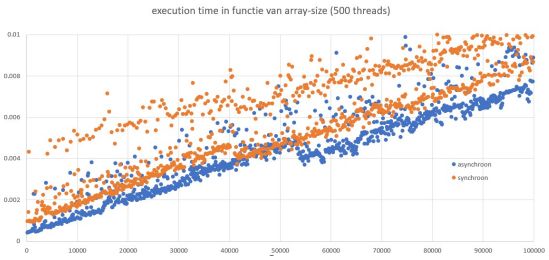


Figure 10: Experiment 2

Zoals te zien is in bovenstaande afbeeldingen presteert het asynchrone geval beter dan het synchrone geval. Naarmate array-size groter wordt zal het verschil tussen de methoden minder merkbaar worden doordat hier de array-processing te dominant wordt in de totale executietijd ten opzichte van de array-creatie.